

Cuentas claras

Arquitectura y decisiones del sistema que dice a Alberto qué facturas pagar. Y que, para cada una, deja por escrito de dónde sale esa respuesta.

Expediente de una factura del lote 1

2026-0811-B_catering.pdf

Proveedor	P005 · Catering Hermanos Pico S.L.
Pedido	P0-2026-0707
Asiento en el ERP	AS-70006 · PENDIENTE
Base e IVA	2.310,00 + 485,10 €
Total facturado	2.920,10 €

No cuadra con el pedido. 2.920,10 € facturados frente a 2.795,10 € pedidos.

No cuadra consigo misma. base más IVA suman 2.795,10 €, no 2.920,10 €.

No cuadra con el ERP. el asiento espera 2.795,10 €.

ESCALAR

13 comprobaciones · 3 discrepancias

Nadie adivinó nada. Las trece comprobaciones de la norma v3 se ejecutaron sobre los campos leídos del PDF y se compararon contra el Excel de pedidos y el asiento del ERP. Tres no pasaron, y las tres van firmadas con su origen.

Decidido en 4,85 ms. Coste en modelos: 0,00 \$, porque este PDF traía texto y no hizo falta mirar la imagen.

500	2,06 s	243	0,00 \$	37
archivos del lote 1, uno por veredicto	el lote completo, en un portátil	archivos por segundo medidos	coste en modelos de esa ejecución	a revisión humana, el 7,4 %
1 · El trabajo de Alberto	p. 2	2.5 Escala y coste	p. 7	
2 · Arquitectura	p. 3	2.6 Lo que aguanta y lo que no	p. 8	
2.1 Las etapas, y lo que dejan escrito	p. 3	2.7 Resiliencia y lote 2	p. 9	
2.2 Estado, datos y reparto	p. 4	3 · ADRs y trade-offs	p. 10	
2.3 La norma v3, regla a regla	p. 5	4 · Lo que no podemos demostrar	p. 12	
2.4 Seguir una decisión	p. 6			

Quién lo usa y qué deja de hacer

1 El trabajo de Alberto

Alberto aprueba pagos. Por cada factura tiene que abrir el PDF, encontrar el pedido en un Excel de catorce hojas, comprobar el NIF y el IBAN contra el maestro de proveedores, sumar base e IVA a mano y entrar en un ERP de 2009 a ver si ese asiento ya está pagado. Seis comprobaciones por documento, y quinientos documentos.

El sistema hace esas seis comprobaciones y entrega dos cosas: un veredicto por archivo (**PAGAR**, **NO_PAGAR** o **ESCALAR**) y, detrás de cada veredicto, el expediente que lo justifica. Alberto no revisa quinientas facturas: revisa treinta y siete, con el motivo y la cifra que no cuadra ya señalados.

Qué salió del lote 1

Veredicto	Archivos	Reparto	Importe	Qué significa para Alberto
PAGAR	452	90,4 %	2.450.391,46 €	Las seis reglas pasan. Se paga sin intervención.
NO_PAGAR	11	2,2 %	19.184,92 €	Pago duplicado demostrado. No es una duda: es un no.
ESCALAR	37	7,4 %	144.119,95 €	Hace falta una persona o un dato corregido.

19.184,92 €
en pagos duplicados que el lote habría dejado pasar
outcomes.jsonl

Lo que el sistema evita, en euros

Los once **NO_PAGAR** no son anomalías difusas. Nueve son facturas cuyo asiento en el ERP ya está en PAGADA: pagarlas sería pagar dos veces. Los otros dos son el mismo pedido facturado dos veces dentro del propio lote ([P0-2026-0492](#), en [2026-0233-A_catering.pdf](#) y [factura_41082.pdf](#)), que sólo se ve mirando el lote entero y no factura a factura.

La cola de revisión, ordenada por lo que cuesta mirarla

Por qué escalan las 37

Motivo	Archivos	Importe	Qué tiene que decidir la persona
iban_mismatch	15	40.063,85 €	El IBAN de la factura no es el registrado. Cambio de banco o fraude.
amount_mismatch	13	69.486,40 €	El total no coincide con el pedido ni con el ERP.
supplier_not_in_master	6	24.623,50 €	Un NIF que no está en el maestro de proveedores.
invalid_date	3	9.946,20 €	Fecha de emisión ilegible o imposible.

El motivo que se muestra es el primero por precedencia, pero el expediente guarda todos los hallazgos: la factura de la portada arrastra tres.

Por qué un proceso por lotes y no una aplicación web

El formato que pide el problema

El trabajo de Alberto llega por lotes: una carpeta de PDFs, de golpe. Así que el producto es un **proceso por lotes con una sola orden**, que es lo que se audita, se reejecuta y se mide. Encima hay una **consola de revisión**, pero no para lanzar el trabajo: para atender las 37 excepciones, abrir el PDF al lado de la regla que falló y ajustar la política sin tocar código. Quinientas facturas no necesitan una pantalla; treinta y siete sí.

La decisión de la que salen todas las demás

2 Arquitectura

Todo el diseño sale de una sola línea: leer un documento es un problema probabilístico, decidir un pago no lo es. Separamos las dos cosas con un contrato de datos en medio, y sólo el lado izquierdo tiene permiso para equivocarse.

Un contrato en medio
`models.py`

Lado probabilístico Extracción Convierte cualquier documento en campos. Es el único sitio donde gastamos modelos, y el único que puede fallar de forma interesante. • 471 PDFs con texto: PyMuPDF y expresiones regulares, seis plantillas. • 29 escaneados: se rinde a la imagen y llama a un modelo de visión. • Caché por SHA-256: el mismo documento no se paga dos veces.	InvoiceData siete campos y una marca de confianza	Lado determinista Decisión Función pura de (campos, datos de referencia, versión de norma). Mismos datos, mismo veredicto, siempre. No lee PDFs ni llama a modelos. • Seis reglas de la norma v3, trece comprobaciones registradas. • Compara contra el Excel de proveedores y pedidos y contra el ERP. • 44 pruebas cubren las ramas sin abrir un solo PDF.
--	--	--

Esta costura contesta la pregunta que importa cuando algo sale mal: ¿leímos mal el documento o aplicamos mal la norma? Son dos capas distintas, con dos formas distintas de arreglarse. Y es lo que permite cambiar de proveedor de modelos sin tocar la lógica de pago.

2.1 Las etapas, y lo que cada una deja escrito

Qué entra, qué hace y qué queda escrito en cada etapa

Etapas	Qué hace	Qué deja escrito	Si falla
0 Datos de referencia <code>business_data · erp_snapshot</code>	Limpia el Excel (11 proveedores, 516 pedidos) y baja el ERP entero una vez: 516 asientos en 26 páginas.	La foto del ERP en SQLite, y una fila por descarga con sus reintentos.	Sin foto del ERP no se decide: el lote se para antes de empezar.
1 Ingesta <code>pipeline_collect_files</code>	Lista la carpeta y descarta repetidos por nombre y por hash del contenido.	La lista de archivos del lote, con su SHA-256.	Un archivo ilegible entra igual, y sale escalado.
2 Extracción <code>HybridExtractor</code>	Texto incrustado si hay; visión si no; línea base si el extractor revienta.	Campos, método, latencia, coste y el fragmento de página de donde sale cada campo.	Cae a la línea base; si tampoco, marca la extracción como no válida.
3 Decisión <code>rules_engine.decide_batch</code>	Aplica la norma versionada y busca pedidos repetidos en el lote y en los anteriores.	Veredicto, motivo, todos los hallazgos, las 13 comprobaciones y la evidencia.	Norma desconocida: error inmediato. Nunca cae a otra versión por su cuenta.
4 Estado y entregable <code>state · deliverables</code>	Guarda la decisión vigente y la histórica, y escribe el JSONL de forma atómica.	Las tres tablas de estado y <code>outcomes.jsonl</code> , validado línea a línea.	Si el volcado falla, el archivo anterior queda intacto: se escribe aparte y se renombra.
5 Consola <code>web/ · Next.js</code>	Muestra la cola de revisión, el expediente de cada archivo y la política editable.	Nada propio: lee y escribe a través de la capa de Python.	Si la consola se cae, el proceso por lotes sigue funcionando solo.

Las etapas 0 a 4 son una orden de terminal y dos ficheros SQLite. No hay cola de mensajes ni servicio que mantener encendido: nada de eso hacía falta para quinientos archivos, y sí hacía falta poder repetir la ejecución y explicarla. Lo que cambiaría con otro volumen está en las páginas 7 y 8.

Dónde vive el estado
state.py

2.2 Estado, datos y reparto del trabajo

El estado operativo vive en SQLite; el JSONL es sólo el formato de entrega. Esa distinción es la que permite reprocesar sin duplicar y contestar preguntas que un fichero de texto no contesta.

Tabla	Clave	Qué guarda	Para qué sirve
decisions	file_id	Veredicto, motivo, hallazgos, evidencia, campos extraídos, método, versión de norma, latencia y coste.	La decisión vigente de cada archivo. Se reescribe por <i>upsert</i> : nunca hay dos.
decision_history	run_id + file_id	Lo mismo, pero inmutable y con el lote al que pertenece.	Comparar ejecuciones y ver qué veredicto cambió al cambiar la norma o el ERP.
runs	run_id	Estado, recuentos, inicio y fin, segundos, archivos por segundo, coste y estadísticas del extractor.	La observabilidad del lote, sin instrumentación aparte.
erp_asientos	asiento_id	Pedido, proveedor, NIF, importe esperado, estado y cuándo se leyó.	Decidir sin volver a llamar al ERP por cada factura.
snapshot_runs	run_at	Total de asientos y estadísticas de la descarga.	Saber de qué foto del ERP sale una decisión.

Quién decide qué

El reparto entre modelos, reglas y personas

Quién	De qué responde	Volumen	Y de qué no responde
Código determinista	Leer los 471 PDFs con texto y decidir las 500 facturas.	500 de 500 decisiones	De interpretar imágenes. Si el PDF no trae texto, no lo intenta.
Modelo de visión	Convertir 29 escaneos en campos, validados contra el esquema.	29 de 500 extracciones	De decidir. Su salida entra en las mismas reglas que todo lo demás.
Persona	Resolver los escalados, corregir campos con una anulación firmada y ajustar la política.	37 de 500 revisiones	De mirar los 452 que cuadran. Ese es justamente el trabajo que se ahorra.

Las correcciones humanas no se editan sobre la base de datos: se declaran en un fichero y se atan al SHA-256 exacto del PDF. Si el documento cambia, la corrección caduca sola y el archivo vuelve a escalar en vez de heredar un arreglo viejo. En el lote 1 hay una ([scan_021.pdf](#)), anotada en la traza como +human-override.

Las seis reglas
rules_engine.py

2.3 La norma v3, regla a regla

La norma de pagos vive en código como seis reglas y trece comprobaciones. Cada comprobación tiene un código, y cada código tiene un veredicto por defecto que Alberto puede cambiar sin que nadie toque el motor.

	Regla	Contra qué se compara	Códigos y veredicto por defecto
0	Extracción completa	Los campos necesarios para poder aplicar las reglas.	incomplete_extraction ESCALAR
1	Proveedor e IBAN	Maestro de proveedores del Excel, por NIF.	supplier_not_in_master, iban_mismatch ESCALAR
2	Pedido e importe	Hoja de pedidos: que exista, que sea de ese proveedor y que el total cuadre.	pedido_not_found, pedido_supplier_mismatch, amount_mismatch ESCALAR
3	Aritmética del IVA	La propia factura: base más IVA, y base por tipo.	total_not_base_plus_iva, iva_miscalculated ESCALAR
4	Fecha	La fecha de referencia de la ejecución.	invalid_date, future_date ESCALAR
5	ERP y duplicados	El asiento del ERP, el resto del lote y los lotes anteriores.	already_paid, duplicate_pedido NO_PAGAR ; el resto ESCALAR

La tolerancia de importes es de un céntimo, y la precedencia cuando hay hallazgos que apuntan a sitios distintos es **ESCALAR** > **NO_PAGAR** > **PAGAR**. El mapa de los 18 códigos a veredictos vive en un fichero de política editable desde la consola: cuando Alberto decida que un IBAN distinto es motivo de rechazo y no de revisión, eso es un cambio de configuración que queda registrado en la ejecución, no un cambio de código.

Qué regla decide, en la práctica

Cuál de las seis decide cada factura

R	Motivo principal	Archivos	Veredicto	Qué lo delata
—	all_rules_pass	452	PAGAR	Las trece comprobaciones pasan.
1	iban_mismatch	15	ESCALAR	El IBAN no es el del maestro de proveedores.
2	amount_mismatch	13	ESCALAR	El total no coincide con el importe del pedido.
5	already_paid	9	NO_PAGAR	El asiento del ERP ya está en PAGADA.
1	supplier_not_in_master	6	ESCALAR	El NIF no está registrado.
4	invalid_date	3	ESCALAR	Fecha de emisión ilegible o imposible.
5	duplicate_pedido	2	NO_PAGAR	El mismo pedido aparece dos veces en el lote.

Sólo siete códigos aparecen como motivo principal en este lote, pero los demás no son código muerto: total_not_base_plus_iva y erp_amount_mismatch aparecen como hallazgos secundarios (la factura de la página 6 arrastra los dos), y los que no se activaron —pedido inexistente, IVA mal calculado, fecha futura, extracción incompleta— están cubiertos por las pruebas del motor. Que un código no se dispare en un lote no significa que no esté defendido.

La factura de la portada,
entera

2.4 Seguir una decisión

Esto es lo que el sistema guarda de `2026-0811-B_catering.pdf`. No es un resumen escrito para este documento: son las trece comprobaciones tal como salen del motor de reglas, cada una con el valor observado, el esperado y la fuente de ese valor esperado.

R	Comprobación	Resultado	Observado	Esperado, y de dónde sale
0	Extracción completa	pasa	texto incrustado	campos suficientes Factura
1	Proveedor registrado	pasa	B96233419 Factura	B96233419 Excel · Proveedores · P005
1	IBAN del proveedor	pasa	ES1800815290070001234567 Factura	ES1800815290070001234567 Excel · Proveedores · P005
2	Pedido existente	pasa	PO-2026-0707 Factura	PO-2026-0707 Excel · Pedidos_2026
2	Proveedor del pedido	pasa	P005 Factura y maestro	P005 Excel · Pedidos_2026 · PO-2026-0707
2	Total frente al pedido	no pasa	2.920,10 € Factura	2.795,10 € Excel · Pedidos_2026 · PO-2026-0707
3	Base más IVA	no pasa	2.920,10 € Factura	2.795,10 € Cálculo: 2.310,00 + 485,10
3	Cálculo del IVA	pasa	485,10 € Factura	485,1000 € Cálculo: 2.310,00 × 21 %
4	Fecha no futura	pasa	2026-05-26 Factura	≤ 2026-09-19 Fecha de referencia de la ejecución
5	Asiento en el ERP	pasa	PO-2026-0707 Factura	PO-2026-0707 ERP · AS-70006
5	Total frente al ERP	no pasa	2.920,10 € Factura	2.795,10 € ERP · AS-70006
5	Estado de pago	pasa	PENDIENTE ERP · AS-70006	PENDIENTE Norma de pagos v3
5	Pedido único en el lote	pasa	1 aparición	1 Lote 1 y lotes anteriores

Lo que va anotado alrededor
decisiones

Y lo que acompaña al veredicto

Veredicto y motivo	ESCALAR · amount_mismatch
Todos los hallazgos	amount_mismatch, total_not_base_plus_iva, erp_amount_mismatch
Evidencia	P005 · PO-2026-0707 · AS-70006 · PENDIENTE
Cómo se leyó	embedded_text, plantilla narrative
Versión de norma	norma-v3
Latencia y coste	4,85 ms · 0,00 \$
Ejecución	2026-09-19T11:20:35Z, con la foto del ERP de esa misma mañana

Qué se ve mientras el lote
corre

Las señales en directo

El proceso emite una línea JSON por suceso: `run_start`, `extracted`, `decided`, `override_skipped`, `data_error`, `data_warnings` y `run_done` con el resumen. La consola consume ese mismo flujo, así que lo que ve Alberto en la pantalla es exactamente lo que se guarda en la base de datos. No hay dos caminos.

El trabajo pendiente tampoco es una métrica aparte: es la consulta `result = 'ESCALAR'`. Hoy devuelve 37 filas y 144.119,95 €.

Medido, no estimado
runs

2.5 Escala y coste

Lo de esta página está separado en dos: lo que hemos medido en una ejecución real y lo que estimamos a partir de esas medidas. Lo segundo va marcado como estimación.

Medida	Valor	De dónde sale
Lote 1 completo, de carpeta a JSONL	2,055 s	Columna elapsed_s de la ejecución.
Rendimiento	243,3 arch./s	Columna files_per_s, con los 29 escaneos ya en caché.
Latencia de extracción por archivo	4,1 ms	Media de 500 filas; mínimo 0,83 ms, máximo 16,64 ms.
Extracción por visión, en frío	5,74 s	Media de los 29 escaneos; mediana 5,22 s, rango de 4,36 a 8,60 s.
Descarga completa del ERP	3,489 s	516 asientos, 26 páginas, 30 peticiones, 3 reintentos.
Coste en modelos de la ejecución	0,00 \$	471 archivos no llaman a ningún modelo; los 29 restantes salieron de caché.
Hardware	1 portátil	Intel Core i5-12500H, 16 hilos, 16 GB, Debian 13, Python 3.13. Un solo proceso.

La fórmula, y por qué el 94 %
del lote sale gratis

Coste por lote

```
coste_lote = n_escaneos_nuevos × precio_visión
471 digitales × 0 $ + 29 escaneos × 0,0003 $ ≈ 0,0087 $ por los 500 archivos
precio_visión ≈ 0,0003 $/página (0,15 $/M tokens de entrada, 0,60 $/M de salida)
```

El término que falta es el que importa: **los documentos con texto no aparecen en la fórmula**, porque nunca llaman a un modelo. El coste no crece con el número de facturas, crece con el número de *escaneos nuevos*. Y como la caché indexa por contenido, reejecutar un lote entero cuesta cero: reprocesar pasa a ser una decisión barata en lugar de una negociación.

A 100.000 facturas con la misma proporción de escaneos (5,8 %): *estimación* de 5.800 llamadas de visión, unos 1,74 \$ en modelos. A esa escala el coste dominante no es el modelo, es el tiempo.

Tiempo a 100.000 facturas

Tramo	Estimación	Cómo se calcula, y qué lo limita
94.200 con texto	≈ 6,5 min	94.200 ÷ 243 arch./s con un solo proceso. Es trabajo por archivo y sin estado compartido: con 8 procesos baja de un minuto.
5.800 escaneos, en serie	≈ 9,2 h	5.800 × 5,74 s. Inaceptable, y es el cuello de botella real del sistema.
5.800 escaneos, 16 en paralelo	≈ 35 min	Las llamadas son independientes; el techo lo pone la cuota del proveedor, no la máquina.
Decidir las 100.000	< 1 min	Función pura en memoria contra dos índices. No es el problema.

La conclusión operativa es que escalar este sistema no es escalar la decisión: es administrar la cuota de visión y repartir la extracción entre procesos. Las dos cosas están aisladas en la etapa 2.

Los techos que ya conocemos

2.6 Lo que aguanta y lo que no

Estos son los límites del diseño actual, con lo que haríamos en cada caso. Ninguno es una sorpresa: son consecuencias aceptadas de haber elegido lo más pequeño que resolvía el problema.

Límite	Cuándo aparece, y qué haríamos
SQLite admite un escritor	En cuanto haya más de un proceso escribiendo. La capa de estado está aislada en un módulo: se cambia por Postgres sin tocar reglas ni extractores.
El ERP tolera 10 peticiones/s, y la sesión caduca a los 15 min o 300 usos	Ya lo esquivamos: se descarga una foto completa con 30 peticiones y las 500 decisiones leen de SQLite. Sin esa foto serían 500 llamadas y una sesión caducada a mitad del lote.
El Excel se carga entero en memoria	Con 516 pedidos es irrelevante. Con millones, pasa a ser una tabla indexada del mismo almacén de estado.
La cuota del proveedor de visión	Es el techo del paralelismo. Se reparte entre varios modelos y proveedores, y la caché absorbe las reejecuciones.

Correos, imágenes, hojas de cálculo

Añadir un tipo de archivo nuevo

El punto de extensión es el contrato `InvoiceData`: un extractor nuevo es una función que devuelve ese objeto, y las reglas no se enteran. Ya hay tres caminos además del PDF (XML de FacturaE y UBL, correos, documentos de texto y hojas de cálculo) detrás de un interruptor en la política que hoy acepta sólo .pdf, para no ampliar la superficie sin necesidad. Aceptar correos en producción es activar ese interruptor y escribir pruebas del extractor nuevo; ni una línea de la lógica de pago cambia. Eso es exactamente lo que compra la costura de la página 3.

Por qué no puede haber pagos duplicados

Idempotencia

- La clave es el `file_id`: reprocesar sustituye la decisión, no añade otra.
- La ingesta descarta repetidos por nombre y por hash del contenido.
- La visión se cachea por SHA-256: el mismo documento no se paga dos veces.
- Los pedidos de lotes anteriores se leen del historial, así que una factura repetida entre lotes se ve.
- El JSONL se escribe en un temporal y se renombra: o está entero o no está.

Qué está comprobado hoy
tests/ · 44 pruebas

Lo que cubren las pruebas

44 pruebas en 2,34 s cubren las ramas del motor de reglas, el aislamiento entre lotes, el contrato del JSONL, las anulaciones atadas al hash, la degradación cuando no hay visión y el bloqueo de una norma no implementada. El motor de reglas se prueba sin abrir un PDF, que era justamente el objetivo de separar las dos capas.

La precisión de lectura se comprobó a mano sobre 45 documentos: 98,4 % de campos correctos, 100 % en los 30 digitales y 95,2 % en los 15 escaneados. Los dos documentos con algún campo mal son escaneos degradados a propósito, y los dos acabaron escalados. Ningún error de lectura detectado produjo un **PAGAR** indebido.

La regla de fondo: fallar en cerrado

2.7 Resiliencia y recuperación

Ningún fallo puede convertirse en un pago. Cuando el sistema no sabe, escala; cuando no puede, se para. Nunca rellena el hueco con algo plausible.

Qué falla	Qué hace el sistema	Qué se conserva	Cómo se recupera
El proveedor de visión no responde	Tres intentos con espera creciente (0, 1 y 2 s) y salto a los modelos alternativos.	Los 471 digitales y los escaneos ya en caché.	Se relanza el lote: sólo se reintentan los escaneos sin resolver.
Caen todos los proveedores	La extracción se marca como no disponible y la regla o no pasa.	Todo lo demás, decidido con normalidad.	Esos archivos salen como ESCALAR y aparecen en la cola. No como pagos.
El modelo o el extractor devuelven algo que no es una factura	La salida se valida contra el esquema y se rechaza; si el extractor revienta, cae a la línea base y lo anota en el método.	Nada contaminado entra en las reglas, y el lote no se interrumpe por un archivo.	Reintento y, si insiste, escalado. Queda marcado para revisar el extractor con ese documento.
El ERP falla como suele fallar	ORA-00600: reintenta la misma llamada, hasta 6 veces. 401: vuelve a autenticarse. 429: espera lo que diga la cabecera, y de salida no pasamos de 8,3 peticiones/s sobre un techo de 10.	Las páginas ya descargadas y el progreso de la sesión.	Automático. Medido: 3 reintentos por ORA-00600 en 30 peticiones, sin intervención.
El ERP no está	El lote no arranca.	La foto anterior del ERP, con su fecha.	Decidir con una foto vieja tiene que ser una elección, no un accidente.
Se interrumpe el proceso	El JSONL se escribe aparte y se renombra al final.	Las decisiones ya guardadas y la caché de visión.	Se relanza. Los hashes evitan pagar dos veces por lo ya leído.
Un PDF corregido a mano cambia	La anulación está atada al SHA-256 y deja de aplicarse.	La anulación, inerte, y el aviso <code>override_skipped</code> .	El archivo vuelve a escalar en vez de heredar una corrección caducada.
Se pide una norma que no existe	Error inmediato. No cae a la versión anterior.	Todo el estado previo.	Se implementa la norma: decidir con la norma equivocada es peor que no decidir.

El lote del sábado y la norma nueva

Qué pasa cuando llega el lote 2

El lote 2 no es una ejecución especial: es otro valor de `--batch`, con su carpeta, su norma y su propia salida. Qué se conserva y qué se reprocesa está decidido de antemano.

Qué cambia	Qué hay que reprocesar, y qué no
40 facturas nuevas	Sólo ellas. El lote 1 no se toca y su fichero de salida no se reescribe.
La norma pasa a v4	El lote 2 se decide con <code>norma-v4</code> en su propio evaluador. La v3 sigue existiendo, así que el lote 1 continúa siendo reproducible.
El ERP se actualiza	Se baja una foto nueva. El lote 2 se niega a arrancar si la foto no declara la actualización cargada.
Una factura repite un pedido del lote 1	Se detecta contra el historial y sale como NO_PAGAR : el caso que sólo se ve si el estado sobrevive entre lotes.
Alberto cambia un dato de la Caja	Se relanza el lote afectado. El historial guarda el veredicto anterior, así que «qué cambió y por qué» es una comparación, no una reconstrucción.

Un ejemplo concreto, con la factura de la página 6

Si el asiento **AS-70006** pasara a PAGADA, la regla 5 fallaría por `already_paid` y el veredicto seguiría siendo **ESCALAR**: hay una duda de importe que un pago duplicado no resuelve. Si además cuadrara el importe, pasaría a **NO_PAGAR**.

Cuatro decisiones que se pueden discutir

3 ADRs y trade-offs

Estas son las cuatro decisiones de las que depende todo lo anterior. Cada una lleva lo que renunciamos a tener, no sólo lo que ganamos.

La costura de la página 3

ADR-01 · aceptada

Separar la lectura del documento de la decisión de pago

Contexto	Las facturas llegan en seis plantillas distintas y algunas son sólo imagen, así que leerlas exige modelos probabilísticos. Decidir un pago, en cambio, tiene que ser reproducible y defendible ante una norma escrita.
Alternativas	1. Un modelo lee la factura y dicta el veredicto en la misma llamada. 2. Sólo expresiones regulares, tanto para leer como para decidir. 3. Normalizar todo a un esquema y decidir después con un motor determinista.
Decisión	La tercera. La extracción produce InvoiceData; el motor de reglas consume InvoiceData y no sabe qué es un PDF. Sólo la extracción tiene permiso para ser probabilística.
Consecuencias	Mantener un esquema canónico cuesta coordinación: cambiarlo toca extractores, persistencia y reglas a la vez. Y una lectura equivocada pero plausible sigue pudiendo arrastrar la decisión, así que la evidencia de origen y el escalado siguen siendo necesarios. Aceptamos las dos cosas.
Evidencia	El motor se prueba sin abrir un PDF. La traza de la página 6 dice, para cada comprobación, si el valor discutido viene de la factura, del Excel o del ERP: ésa es exactamente la pregunta que esta separación deja contestar.

Por qué 37 y no 48

ADR-02 · aceptada

Escalar la duda, y reservar el «no» para lo demostrado

Contexto	Pagar una factura dudosa cuesta dinero. Rechazar toda anomalía bloquea pagos legítimos y desgasta la confianza en el sistema. La norma pide explícitamente revisión humana cuando hay duda razonable.
Alternativas	1. Toda comprobación fallida es NO_PAGAR. 2. Un modelo resuelve los casos ambiguos. 3. ESCALAR para la duda, y NO_PAGAR sólo para el pago duplicado demostrable.
Decisión	La tercera, con precedencia ESCALAR > NO_PAGAR > PAGAR y un mapa de códigos a veredictos configurable. Descartamos la segunda a propósito: un modelo que adjudica casos límite es justo la parte que no podríamos defender ante un auditor.
Consecuencias	La cola de revisión es más larga de lo estrictamente necesario, y la precedencia puede devolver ESCALAR aunque otro hallazgo, por sí solo, ya diga que no se pague. Sale a cuenta si la consola enseña el desajuste exacto, y por eso la consola no es un adorno: es la contrapartida de esta decisión.
Evidencia	37 escalados frente a 11 rechazos en el lote 1. Los 11 se apoyan en un estado PAGADA del ERP o en un pedido repetido dentro del lote, no en una valoración. Y ningún error de lectura de los detectados en la revisión manual produjo un PAGAR indebido.

29 de 500

ADR-03 · *aceptada*

Extracción híbrida: visión sólo cuando hace falta, y en caché

Contexto	471 de los 500 PDFs traen texto incrustado y 29 son sólo imagen. Mandar los 500 a un modelo añade latencia, coste y una dependencia externa a todo el lote. Limitarse al texto deja 29 facturas sin procesar, y entre ellas hay pagos reales.
Alternativas	1. Todo por visión, por uniformidad. 2. Sólo texto incrustado, y las imágenes fuera de alcance. 3. Texto primero, visión como último recurso, y caché por contenido.
Decisión	La tercera. El umbral es objetivo (100 caracteres útiles de texto), la salida del modelo se valida contra el esquema, el resultado se guarda indexado por el SHA-256 del PDF y hay una cadena de modelos alternativos por si el primero no responde. Si no se puede leer con garantías, se marca así y escala.
Consecuencias	Hay dos caminos de lectura que deben producir el mismo contrato, y la caché hay que invalidarla cuando cambien el <i>prompt</i> o el esquema. Y la precisión de la ruta de visión es medible pero no perfecta: escaneos degradados a propósito siguen produciendo campos dudosos, que por diseño acaban escalados y no pagados.
Evidencia	94,2 % del lote resuelto sin modelos. 98,4 % de campos correctos sobre 45 documentos etiquetados a mano (100 % en los 30 digitales, 95,2 % en los 15 escaneados). Coste del lote: 0,0087 \$ a precio de lista y 0,00 \$ medido con caché. La ejecución de visión usó google/gemini-3.8-flash a través de Vercel AI Gateway, con anthropic/claude-sonnet-4-5 y openai/gpt-4o configurados como alternativas.

Una decisión vigente por
archivo

ADR-04 · *aceptada*

Guardar el estado en SQLite, y tratar el JSONL como entrega y no como verdad

Contexto	Hay que saber la decisión vigente de cada archivo, conservar las métricas de cada ejecución y poder reprocesar cuando cambien la norma o los datos de origen. Y la consola y el proceso por lotes tienen que ver lo mismo, sin montar infraestructura para un fin de semana.
Alternativas	1. Sólo los ficheros JSONL generados. 2. Ficheros JSON sueltos, uno por archivo. 3. Una base de datos gestionada, con su despliegue. 4. SQLite local, propiedad exclusiva de la capa de Python.
Decisión	La cuarta: runs, decisions con file_id como clave primaria y decision_history inmutable. El JSONL se genera desde ahí. La consola no abre el fichero SQLite: pasa por la misma capa de Python, para que haya un solo dueño del estado.
Consecuencias	Un solo escritor, así que en cuanto haya varios trabajadores hay que migrar; lo asumimos porque la capa de estado está aislada y la migración es local. La tabla de decisiones vigentes por sí sola no guarda todas las versiones de una factura, y de ahí el historial. Y una recuperación más fina a mitad de lote exigiría persistir el estado por archivo antes de terminar, que hoy no hacemos.
Evidencia	Las métricas de la página 7 son columnas de la tabla runs, no un cálculo hecho para este documento. La cola de revisión es una consulta. El «qué cambió» entre normas es una comparación entre dos filas del historial.

Mejor aquí que en una pregunta

4 Lo que no podemos demostrar

Límite	Qué implica
Los datos son sintéticos y no tenemos la referencia privada	Podemos demostrar que cada decisión es consistente con la norma y con sus fuentes. No podemos demostrar que coincide con la clave del jurado.
La precisión de lectura sale de 45 documentos etiquetados por una persona	Es una comprobación por muestreo, no una prueba. El 98,4 % hay que leerlo con ese tamaño de muestra delante.
Un solo nodo y un solo escritor	Sobra para 500 y aguanta 100.000 en un rato. No sirve para varios trabajadores en paralelo sin migrar el almacén.
El coste de visión es precio de lista	La ejecución medida costó 0,00 \$ por caché y cuota gratuita. La fórmula sirve; el precio unitario es una estimación.
La recuperación es por lote, no por archivo	Relanzar cuesta segundos y cero dólares, así que no hemos invertido en un reanudado más fino. A otra escala sería lo primero que arreglaríamos.

Lo que presentamos como mejora adicional

5 La consola de revisión

El proceso por lotes ya cumple el encargo: 500 veredictos con su expediente. La mejora que presentamos aparte es lo que pasa *después* del veredicto, que es el trabajo que de verdad ocupa a Alberto.

Qué añade	Qué le resuelve a Alberto
La cola de revisión con el PDF al lado	Abre el documento junto a la comprobación que falló, con el valor observado y el esperado. Un escalado deja de ser un aviso y pasa a ser una tarea que se puede cerrar.
La política editable	Cambiar la tolerancia, la fecha de referencia o el veredicto de un código es un formulario, no un despliegue. Y queda registrado en la ejecución.
La pantalla de indicadores	Facturas resueltas sin tocar, euros por veredicto, fuga de pago doble evitada, motivos más frecuentes y proveedores que más escalan. 19.184,92 € y 15 IBANes discrepantes son una conversación con el proveedor, no una métrica.
El historial y el enlace directo	Cada archivo tiene su URL. «Enséñame la factura 2026-0811-B» es un enlace, incluso en mitad de la defensa.

Dónde mirar para cada pregunta

Y si el jurado pregunta

Pregunta	Qué se enseña
Seguidme una decisión real	La traza de la página 6, y esa misma factura en la consola.
¿Qué pasa si se cae el proveedor de modelos?	La tabla de fallos de la página 9, y se corta la clave en directo: el lote termina y esos archivos escalan.
¿Cuánto cuesta y cuánto tarda?	La fórmula y las medidas de la página 7, con las estimaciones marcadas como tales.
¿Y si cambia un dato de la Caja?	Se relanza el lote y se compara con el historial: qué veredicto cambió, y por qué.

Quinientas facturas decididas en dos segundos importan menos que poder explicar una sola de ellas, dos meses después, delante de quien pregunta.